

Graphs

Dijkstra's shortest-path algorithm.

procedure `dijkstra`(G, l, s)

Input: Graph $G = (V, E)$, directed or undirected;
positive edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u **reachable** from s , `dist`(u) is set
to the distance from s to u .

for all $u \in V$:

`dist`(u) = ∞

`prev`(u) = nil

`dist`(s) = 0

$H = \text{makequeue}(V)$ (using `dist`-values as keys)

while H is not empty:

$u = \text{deletemin}(H)$

 for all edges $(u, v) \in E$:

 if `dist`(v) > `dist`(u) + $l(u, v)$:

`dist`(v) = `dist`(u) + $l(u, v)$

`prev`(v) = u

`decreasekey`(H, v)

We can't decrease key of the
node which was deleted before

Observations

At the end of each iteration of the while loop, the following conditions hold: (1) there is a value d such that all nodes in R are at distance $\leq d$ from s and all nodes outside R are at distance $\geq d$ from s , and (2) for every node u , the value $\text{dist}(u)$ is the length of the shortest path from s to u whose intermediate nodes are constrained to be in R (if no such path exists, the value is ∞).

Not the final!

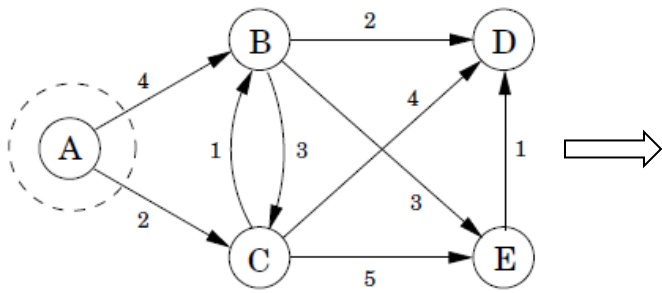
Time complexity

Once 'makequeue' takes $|V|$ time. $|V|$ times 'deletemin' takes $|V|^2$ time*. $|E|(2|E|)$ times 'decreasekey' takes $|E|$ time* . * array based priority queue.

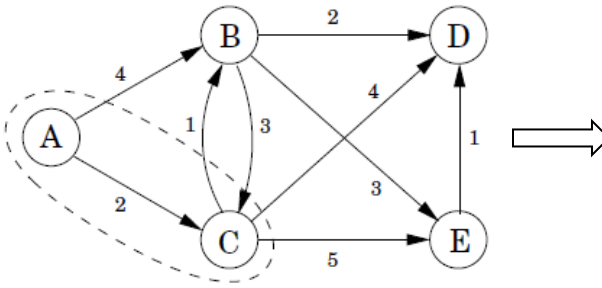
Additional Space complexity: $|V|$

Sparse or Dense?

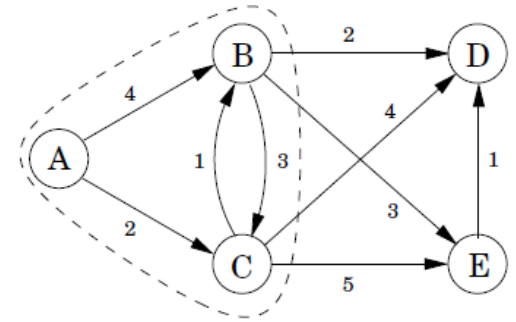
Implementation	deletemin	insert/ decreasekey	$ V \times \text{deletemin} + E \times \text{insert}$
Array	$O(V)$	$O(1)$	$O(V ^2)$
Binary heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
d -ary heap	$O(\frac{d \log V }{\log d})$	$O(\frac{\log V }{\log d})$	$O((V \cdot d + E) \frac{\log V }{\log d})$
Fibonacci heap	$O(\log V)$	$O(1)$ (amortized)	$O(V \log V + E)$



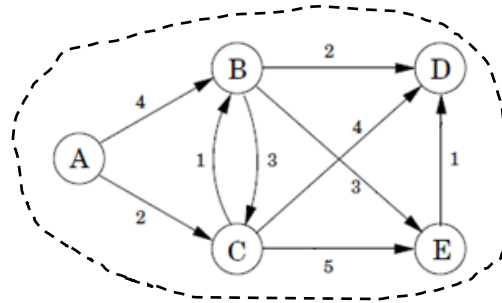
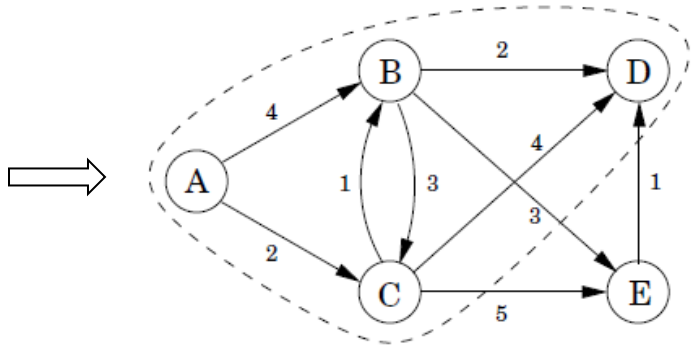
A: 0	D: ∞	B:A	D:[]
B: 4	E: ∞		
C: 2		C:A	E:[]



A: 0	D: 6	B:C	D:C
B: 3	E: 7		
C: 2		C:A	E:C

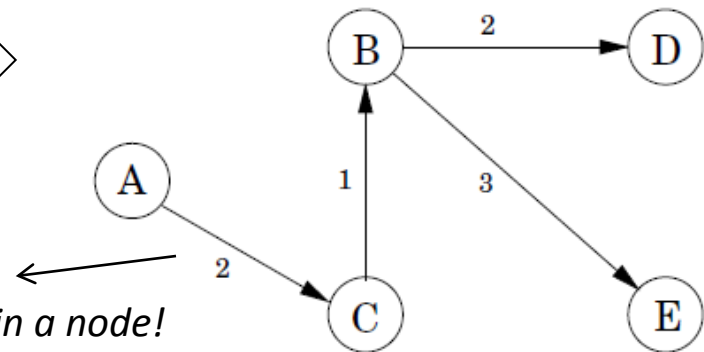
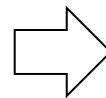


A: 0	D: 5	B:C	D:B
B: 3	E: 6		
C: 2		C:A	E:B



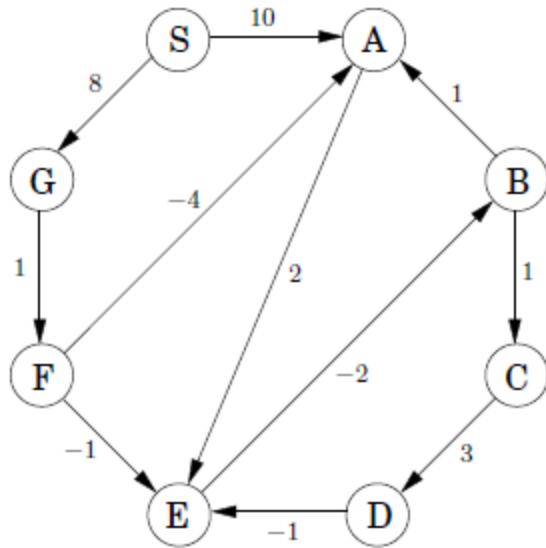
B: 3,C	D: 5,B
C: 2,A	E: 6,B

A: 0	D: 5	B:C	D:B
B: 3	E: 6		
C: 2		C:A	E:B

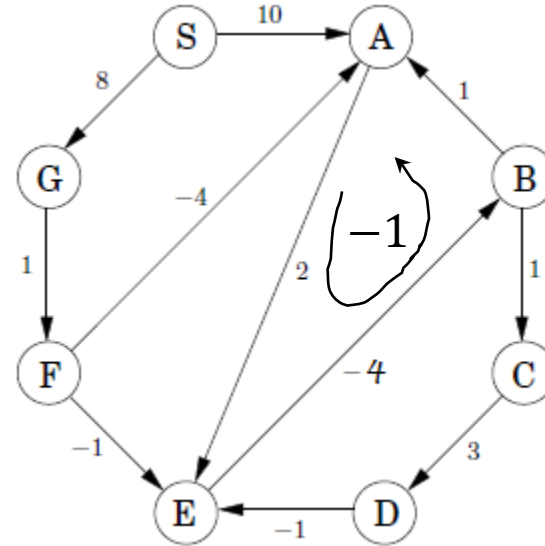


Polytree! Otherwise will need an extra entry in a node!

Shortest path with negative edge lengths:



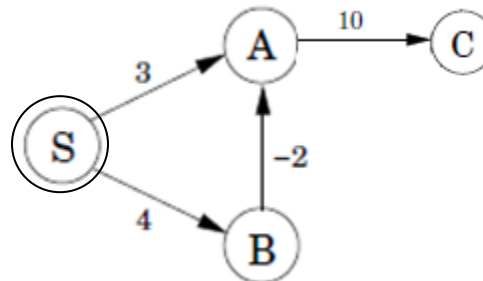
Negative edges, but no negative cycles: **Good!**



Negative cycles: ill-defined (shortest path)!

Consider this

S to A is 2 via B, but S to A is smaller than S to B!



Dijkstra's development of known node set does not hold.

```

H = makequeue(V)
while H is not empty:
    u = deletemin(H)
    for all edges (u,v) ∈ E:
        if dist(v) > dist(u) + l(u,v):
            dist(v) = dist(u) + l(u,v)
            prev(v) = u
            decreasekey(H, v)

```

procedure update((u,v) ∈ E)
 $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u,v)\}$

Ensures $\text{dist}(v) \leq \text{dist}(u) + l_{uv}$

1. If $\text{dist}(u)$ is TRUE and u is the second last node in the shortest path to v , then $\text{dist}(v)$ is TRUE.

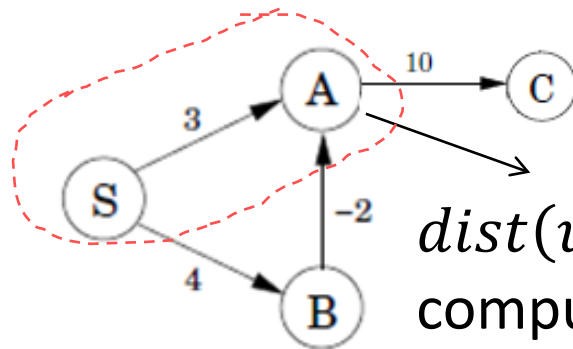
Actual! 2. $\text{dist}(v)$ can never be smaller than its TRUE value. So any number of irrelevant such updates do not matter. Update *safe*!

How to ensure?

If $dist(u)$ is TRUE and u is the second last node in the shortest path to v , then $dist(v)$ is TRUE.

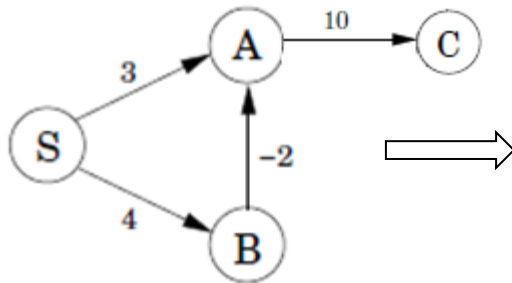
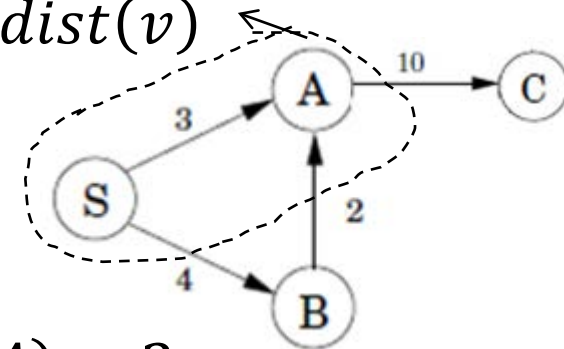
In Dijkstra's: ordered!

If u is the second last node in the shortest path to v that is being used to compute $dist(v)$ then $dist(u)$ is TRUE, that is, $u \in R$



$v = C$
 $u = A$
 $dist(u)$ is true while computing $dist(v)$

$dist(u)$ may not true while computing $dist(v)$



Update (S, A) : $dist(A) = 3$

Update (S, B) : $dist(B) = 4$

Update (A, C) : $dist(C) = 13$

Dijkstra stop

Update (B, A) : $dist(A) = 2$

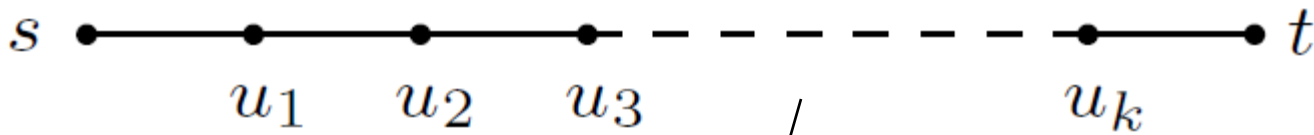
Repeat (can be in arbitrary order):

Update (S, A) : Update (S, B) : Update (A, C) : Update (B, A) :

7 $dist(A) = 2$ $dist(B) = 4$ $dist(C) = 12$ $dist(A) = 2$

Basically, we ignored whether a node is in known set or not. We repeated updates till no changes occur. Will this converge in general?

node t and look at the shortest path to it from s



at most $|V| - 1$ edges

Updates: $(s, u_1), (u_1, u_2), (u_2, u_3), \dots, (u_k, t)$

In that order, but may not be consecutively.. will yield TRUE $dist(t)$!

We will definitely get the required order if we run updates for all the edges $(|E|)$, $|V| - 1$ times!

“Unnecessary
Update safe”

-- Solution!

Time complexity: $O(|E| \cdot |V|)$

Bellman-Ford Algorithm

The Bellman-Ford algorithm for single-source shortest paths in general graphs.

procedure `shortest-paths`(G, l, s)

Undirected??

Input: Directed graph $G = (V, E)$;

edge lengths $\{l_e : e \in E\}$ with no negative cycles;

vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set to the distance from s to u .

for all $u \in V$:

$\text{dist}(u) = \infty$

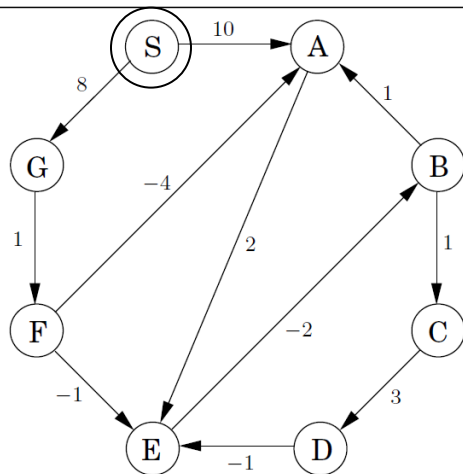
$\text{prev}(u) = \text{nil}$

$\text{dist}(s) = 0$

repeat $|V| - 1$ times:

for all $e \in E$:

update(e)



	Iteration							
Node	0	1	2	3	4	5	6	7
S	0	0	0	0	0	0	0	0
A	∞	10	10	5	5	5	5	5
B	∞	∞	∞	10	6	5	5	5
C	∞	∞	∞	∞	11	7	6	6
D	∞	∞	∞	∞	∞	14	10	9
E	∞	∞	12	8	7	7	7	7
F	∞	∞	9	9	9	9	9	9
G	∞	8	8	8	8	8	8	8

How to detect negative cycles?

Bellman-Ford Algorithm at the $|V|^{th}$ iteration:

Any change means we are working on a graph with negative cycles!

Why?

- From Bellman-Ford, if there are no negative cycles nothing will change in the $|V|^{th}$ iteration

But, is the converse true? That is, if nothing changes in $|V|^{th}$ iteration, are there no negative cycles?

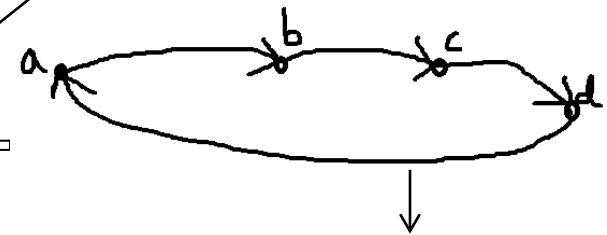
WLOG:



We have:

$$dist(b) \leq dist(a) + l_{ab}$$

$$dist(a) \leq dist(b) + l_{ba}$$



No negative cycle if sum of the edge lengths between arbitrary number of edges is non-negative.

All TRUE dists!

Shortest path on DAG: Time complexity: $O(|E| + |V|)$

Remember: All edges directed in topological order (left to right)

So all shortest paths will have edges that obey the topological order!

A single-source shortest-path algorithm for directed acyclic graphs.

procedure dag-shortest-paths (G, l, s)

Input: Dag $G = (V, E)$;

edge lengths $\{l_e : e \in E\}$; vertex $s \in V$

Output: For all vertices u reachable from s , $\text{dist}(u)$ is set to the distance from s to u .

for all $u \in V$:
 $\text{dist}(u) = \infty$
 $\text{prev}(u) = \text{nil}$

longest paths

↙
just negate all edge lengths

$\text{dist}(s) = 0$

Linearize G

for each $u \in V$, in linearized order:

 for all edges $(u, v) \in E$:

 update (u, v)

The sequence of updates includes every shortest path as a subsequence

Greedy Algorithms

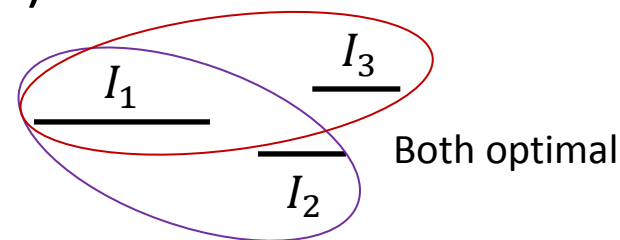
Example: A Scheduling Problem

Input: A set of intervals $A = \{(x_1, y_1), (x_2, y_2), \dots, \dots\}$

$$x_i \leq y_i, x_i, y_i \in \mathbb{R}$$

Output: Schedule $S \subseteq A$ Non-overlapping
(null intersection)

Goal: Maximize $|S|$



Algo Skeleton:

1. Set $S = \phi$
2. While $\exists (x, y) \in A \setminus S$ that does not overlap with any $(x, y) \in S$
 - Choose such an **optimal** $(x^*, y^*) \in A \setminus S$ and add to S
3. Return S

Strategy?!

Greedy because it does not revisit its decisions [here, choosing optimal interval]

Strategy:

Pick the non-intersecting interval with the earliest end time!

How to prove the optimality?

- Let $G = (g_1, g_2, \dots)$ be the greedy solution.
- Let $S = (s_1, s_2, \dots)$ be any optimal solution.

Sorted by end time

Complexity? $n = |A|$
 $O(n \log n) + O(n)$

Lets compare G and S:

Say, $g_1 = s_1$ but $g_2 \neq s_2$

Consider: $S' = (g_1, g_2, s_3, s_4, \dots)$

Is this a valid solution?

g_2 does not intersect with the intervals before it

Does g_2 go with intervals after it?



g_2 was chosen when s_2, s_3 and all other options were available.



Now if s_2 co-exist with all others that follow it, g_2 with **earlier or same end time** can also!

So $S' = (g_1, g_2, s_3, s_4, \dots)$ is valid (optimal).



Like this we can obtain more optimal solutions and end with the greedy solution without ever losing optimality.

Formal proof by contradiction:

Suppose G is NOT optimal.

Pick an optimal S which is closest ($\max |S \cap G|$) to G .

$$S \neq G$$

We can modify S to S' so that:

$$S' \text{ is also optimal \& } |S' \cap G| \geq |S \cap G|$$



Contradiction!